MINISTRY OF EDUCATION
IMAM ABDULRAHMAN BIN
FAISAL UNIVERSITY
COLLEGE OF COMPUTER SCIENCE
& INFORMATION TECHNOLOGY

وزارة التعليم
جامعة الإمام
عبد الرحمن بن فيصل
كلية علوم الحاسب
وتقنية المعلومات

جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

ARTI 502

Deep Learning: 2nd Assignment

Training-Testing CNN with CIFAR Dataset using PyTorch

Year 2026 Term 1

Group #1

Group Members:

#	Member Name	Member Academic ID
1	Ali Walled Al-Hashim	2230004042
2	Hadi Muneer Al-Abu Allairat	2230005761
3	Hadi Osama Al-Shalabi	2230009003
4	Omar Mohammed Hejazi	2230009001
5	Raed Shafiq Almozel	2230001793

Overview

In this assignment, we will have you train some convolutional neural networks and will use a small dataset (CIFAR).

Objective

To implement two core building blocks to deep learning:

- Convolution layers
- Maxpooling layers.

Tasks (with PyTorch)

Task 1: Getting the CIFAR Dataset

Instructions:

- Download the dataset first. Since PyTorch hosts the CIFAR dataset, you can load it using their helper function later.
- You need to change DATA_PATH to an empty directory to download the dataset to.

Deliverable:

Screenshot showing the dataset loaded successfully.

```python
DATA_PATH = '/tmp/cifar10_data'
os.makedirs(DATA_PATH, exist_ok=True)

kaggle_copy = glob.glob('/kaggle/input/**/cifar-10-python.tar.gz', recursive=True)
if kaggle_copy:
    shutil.copy(kaggle_copy[0], DATA_PATH)

train_data = torchvision.datasets.CIFAR10(DATA_PATH, train=True, download=True)
test_data = torchvision.datasets.CIFAR10(DATA_PATH, train=False, download=True)
classes = train_data.classes

print('The dataset loaded successfully')
print('Training images:', len(train_data))
print('Test images:', len(test_data))
print('Classes:', classes)
```

Python

```
The dataset loaded successfully
Training images: 50000
Test images: 10000
Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
```

Task 2: Defining the Network

Instructions:

- Just like with MNIST in the first assignment you need to define your network architecture. This time you will be using convolutional layers and maxpooling to extract features from our images before you feed those features into your final classifier.

- Check out the documentation for nn.Conv2d

- So, the first parameter is the number of channels in the input. Second is the number of filters we'll use (AKA number of channels in the output). Third is kernel size. Next is stride and padding which are optional and have default values.

- Create your training and test functions

Deliverable:

- Provide screenshots of your code to define your network

```
backbone = timm.create_model('convnext_large.fb_in22k', pretrained=True, num_classes=10)
convnext_cifar10 = nn.Sequential(nn.Upsample(size=224, mode='bicubic', align_corners=False), backbone)
```

*Pre-trained convolutional model

- Show screenshots of your training and test functions.

```
def test_model(runner, test_loader):
    runner.eval()
    total_loss = 0
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)
            with torch.amp.autocast('cuda'):
                outputs = runner(images)
            total_loss += F.cross_entropy(outputs, labels, reduction='sum').item()
            correct += (outputs.argmax(1) == labels).sum().item()
            total += images.size(0)

    return total_loss / total, 100 * correct / total
```

```python
def train_one_epoch(model, runner, ema_model, train_loader, optimizer, scaler, settings, step, use_sam):
    model.train()
    steps_per_epoch = len(train_loader)
    total_steps = settings['epochs'] * steps_per_epoch
    total_loss = 0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        for group in optimizer.param_groups:···

        # Forward and backward pass
        with torch.amp.autocast('cuda'):
            outputs = runner(images)
            loss = F.cross_entropy(outputs, labels, label_smoothing=settings['label_smoothing'])
        scaler.scale(loss).backward()

        do_update = True
        if use_sam:···

        # Update the weights
        if do_update:
            scaler.unscale_(optimizer)
            nn.utils.clip_grad_norm_(model.parameters(), 5.0)
            scaler.step(optimizer)
            scaler.update()
            optimizer.zero_grad(set_to_none=True)

        update_ema(ema_model, model, step, total_steps)
        step += 1

        total_loss += loss.item() * images.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
        total += images.size(0)

    return total_loss / total, 100 * correct / total, step
```

Activate Windows

Task 3: Loading Data

Instructions:

- It's worthwhile to check out how the Dataloader loads the images into tensors. You can print out the size of the loaded data in the images tensors. Also, you can use matplotlib to see the visualization of the images data

Deliverable:

- Provide screenshots how to find:
  1. Images tensor size

```python
plain_data = torchvision.datasets.CIFAR10(DATA_PATH, train=True, transform=T.ToTensor())
plain_loader = DataLoader(plain_data, batch_size=128, shuffle=True)

images, labels = next(iter(plain_loader))

print('Images tensor size:', images.shape)
print('Labels tensor size:', labels.shape)
print('Pixel values go from', images.min().item(), 'to', images.max().item())
```

Python

```
Images tensor size: torch.Size([128, 3, 32, 32])
Labels tensor size: torch.Size([128])
Pixel values go from 0.0 to 1.0
```

  2. Data images visualization using matplotlib

Add Code Cell

```python
plt.figure(figsize=(10, 5))
for i in range(32):
    plt.subplot(4, 8, i + 1)
    plt.imshow(images[i].permute(1, 2, 0))
    plt.title(classes[labels[i]], fontsize=9)
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Pyt



Task 4: Dataset Augmentation

Instructions:

- In this assignment, you may find that data augmentation helps you a lot (or possibly hurts your performance).
- Play around with various data augmentations we will suggest some:

1. ToPILImage - useful for many built-in transforms which expect PIL images.
2. RandomHorizontalFlip
3. RandomResizedCrop
4. ColorJitter
5. RandomRotation
6. Normalize
7. Adding various types of noise
8. ToTensor - PyTorch expects the output from the dataset to be a tensor in CxHxW format.

Note: You should be careful about which of these you apply to the test data. You usually don't want to apply noise to the test data, but you do want to normalize it the same way for example.

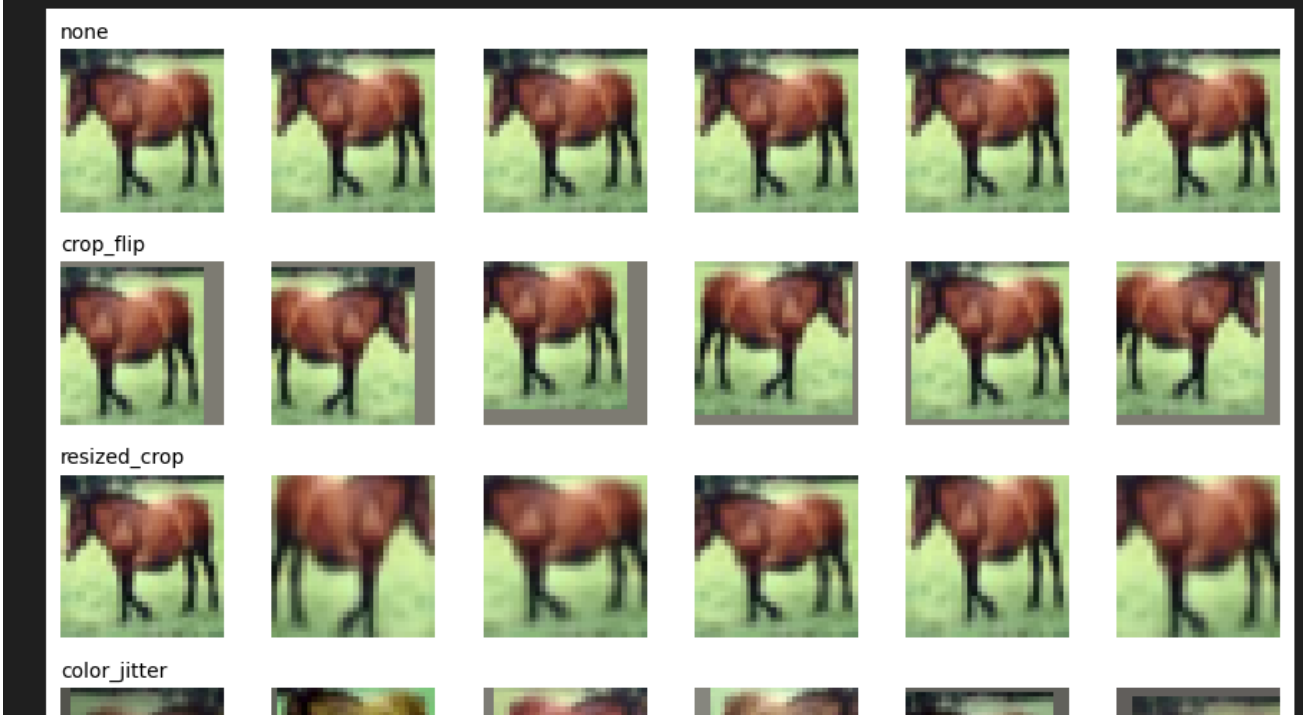
Deliverables:

- Apply your code with various Dataset Augmentation as mentioned above.

```
def make_augmentations(mean, std):
    crop_flip = [T.RandomCrop(32, padding=4, fill=FILL), T.RandomHorizontalFlip()]
    to_tensor = [T.ToTensor(), T.Normalize(mean, std)]
    return {
        'none': T.Compose(to_tensor),
        'crop_flip': T.Compose(crop_flip + to_tensor),
        'resized_crop': T.Compose([T.RandomResizedCrop(32, scale=(0.6, 1.0)), T.RandomHorizontalFlip()] + to_tensor),
        'color_jitter': T.Compose(crop_flip + [T.ColorJitter(0.4, 0.4, 0.4, 0.1)] + to_tensor),
        'rotation': T.Compose(crop_flip + [T.RandomRotation(15, fill=FILL)] + to_tensor),
        'gaussian_noise': T.Compose(crop_flip + [T.ToTensor(), GaussianNoise(0.05), T.Normalize(mean, std)]),
        'salt_pepper_noise': T.Compose(crop_flip + [T.ToTensor(), SaltPepperNoise(0.05), T.Normalize(mean, std)]),
        'trivial_cutout': T.Compose(crop_flip + [T.TrivialAugmentWide(fill=FILL)] + to_tensor + [Cutout(16)]),
    }
```

```python
augmentations = make_augmentations(mean, std)
sample_image = T.ToPILImage()(train_data.data[7])
mean_tensor = torch.tensor(mean).view(3, 1, 1)
std_tensor = torch.tensor(std).view(3, 1, 1)

plt.figure(figsize=(9, 12))
row = 0
for name, transform in augmentations.items():
    for column in range(6):
        augmented = transform(sample_image)
        augmented = (augmented * std_tensor + mean_tensor).clamp(0, 1)
        plt.subplot(len(augmentations), 6, row * 6 + column + 1)
        plt.imshow(augmented.permute(1, 2, 0))
        plt.axis('off')
        if column == 0:
            plt.title(name, fontsize=10, loc='left')
    row += 1
plt.tight_layout()
plt.show()
```



Task 5: Training Function and Tuning hyperparameters

Instructions:

- Generally, it is useful to see how your training is going. Often people print their loss to make sure it goes down and their accuracy to make sure it goes up. But pictures are better than words. So, for this part, you should record and plot the training loss, test loss, and test accuracy (and whatever else you want).
- Try different network architectures and experiment with hyperparameters.

Deliverables:

- Screenshot of your python source code that contains your training function with above requirements

```
def train_model(model, train_loader, test_loader, settings, final_run=False, checkpoint=None):
    model = model.to(device)
    ema_model = copy.deepcopy(model)

    # Split batches between the GPUs
    runner = nn.DataParallel(model) if NUM_GPUS > 1 else model
    ema_runner = nn.DataParallel(ema_model) if NUM_GPUS > 1 else ema_model

    decay_params = [p for p in model.parameters() if p.ndim > 1]
    other_params = [p for p in model.parameters() if p.ndim <= 1]
    optimizer = torch.optim.AdamW([{'params': decay_params, 'weight_decay': settings['weight_decay']},
                                   {'params': other_params, 'weight_decay': 0.0}], lr=settings['lr'])
    scaler = torch.amp.GradScaler('cuda')

    history = {'train_loss': [], 'train_acc': [], 'test_epoch': [], 'test_loss': [], 'test_acc': []}
    start = time.time()
    step = 0
    first_epoch = 1
    earlier_minutes = 0

    if checkpoint is not None: ···

    for epoch in range(first_epoch, settings['epochs'] + 1):
        train_loader.generator.manual_seed(settings['seed'] * 100003 + epoch - 1)
        sam_start = settings['sam_start_epoch']
        use_sam = sam_start is not None and epoch > sam_start

        train_loss, train_acc, step = train_one_epoch(model, runner, ema_model, train_loader, optimizer, scaler,
                                                      settings, step, use_sam)
        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)

        last_epoch = epoch == settings['epochs']
        out_of_time = final_run and time.time() - notebook_start > STOP_HOURS * 3600

        if epoch % 2 == 0 or last_epoch or out_of_time: ···

        if final_run and (epoch % 5 == 0 or last_epoch or out_of_time): ···

        if out_of_time and not last_epoch: ···

    history['minutes'] = earlier_minutes + (time.time() - start) / 60
    return ema_model, history
```

- What design that you tried worked the best? This includes things like network design, learning rate, batch size, number of epochs, and other optimization parameters, data augmentation etc.

  - Network design, we tried:
    - Training from scratch: ~97.5% test accuracy.
    - Other pre-trained models:
      - tf_efficientnetv2_m,
      - convnext_base,
      - convnext_tiny,
      - convnext_large (best performance)

  - learning rates:
    - 3e-5,
    - 1e-4 (best performance),
    - 3e-4

  - weight_decay: 0.01 (best performance), 0.05

```
search_space = {
    'lr': [3e-5, 1e-4, 3e-4],
    'weight_decay': [0.01, 0.05],
    'label_smoothing': [0.0, 0.1],
    'drop_path': [0.0, 0.2],
}
```

  - Batch size fixed at 128 → how many images the VRAM can hold at once.

- What was the final train loss? Test loss? Test Accuracy? Provide the plots for train loss, test loss, and test accuracy.

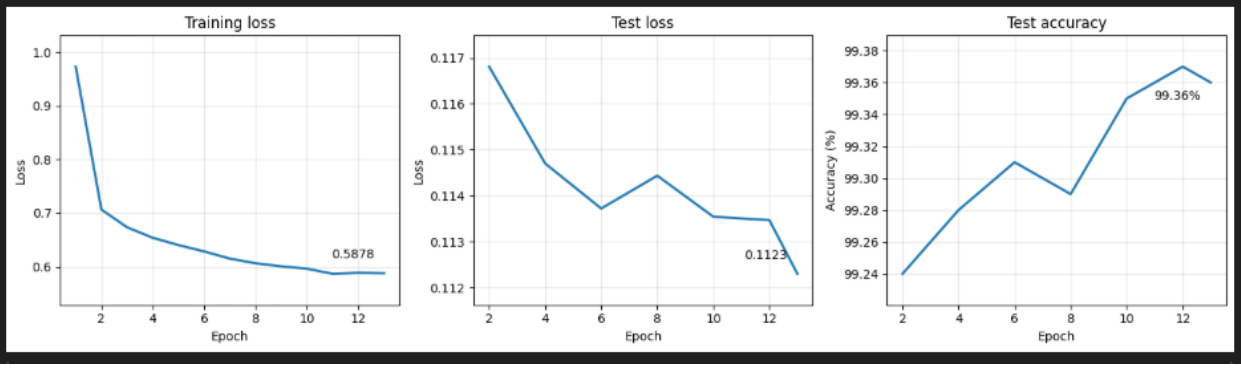
```
train_loader, test_loader = make_loaders(final_settings['augmentation'], final_settings['batch_size'], False, final_settings['model'])

final_model, history = train_model(final_network, train_loader, test_loader, final_settings, final_run=True, checkpoint=checkpoint)

finished = len(history['train_loss']) == final_settings['epochs']
print(f'Training took {history["minutes"] / 60:.2f} hours')
print('Finished all the epochs:', finished)
```

```
epoch 2/13  train loss 0.7066  train acc 91.11  test loss 0.1168  test acc 99.24  time 22.9 min
epoch 4/13  train loss 0.6539  train acc 93.12  test loss 0.1147  test acc 99.28  time 45.6 min
epoch 6/13  train loss 0.6285  train acc 94.24  test loss 0.1137  test acc 99.31  time 68.3 min
epoch 8/13  train loss 0.6064  train acc 95.31  test loss 0.1144  test acc 99.29  time 91.0 min
epoch 10/13  train loss 0.5963  train acc 95.69  test loss 0.1135  test acc 99.35  time 113.6 min
epoch 12/13  train loss 0.5885  train acc 96.08  test loss 0.1135  test acc 99.37  time 146.6 min
epoch 13/13  train loss 0.5878  train acc 96.17  test loss 0.1123  test acc 99.36  time 168.4 min
Training took 2.81 hours
Finished all the epochs: True
```

```
plot_history(history)
```



Task 6: Save your source code

Instructions:

- Save your python source code

Deliverable:

- Submit your GitHub link of your source code along with this document, which includes all screenshots via Blackboard.

-

- End of Assignment -

Grading Rubric

Criterion	Excellent	Good	Developing	Poor	Marks
Task 1: Getting the CIFAR Dataset	Dataset loaded correctly with clear screenshot showing successful execution	Loaded correctly but screenshot is unclear	Dataset loaded with issues or incomplete evidence	Not loaded / missing screenshot	**0.5**
Task 2: Defining the Network	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not answered / incorrect / missing evidence	**2**
Task 3: Loading Data	All clear screenshots	Minor mistakes or missing small details	Several missing descriptions or unclear screenshots	Not loaded / missing screenshot	**0.5**
Task 4: Dataset Augmentation	All datasets augmentation implemented	Major datasets augmentation implemented	Minor datasets augmentation implemented	Not done	**2**
Task 5: Training Function and Tuning parameters	Tuning parameters and model architecture created, screenshot included, and results description explained accurately	Tuning Parameters and model architecture correct but results description is weak	Tuning Parameters and model architecture correct but incorrect results description	Not done	**2**
Task 6: Save your source code	Source code save successfully	Source code save successfully done but minor missing item	Source code save incomplete or file format incorrect	Not submitted / missing	**0.5**
Total					**7.5**